

计算机组成原理课程 设计指导书

张晓彤 刘宏岚 张磊

计算机专业实验室

2018.7

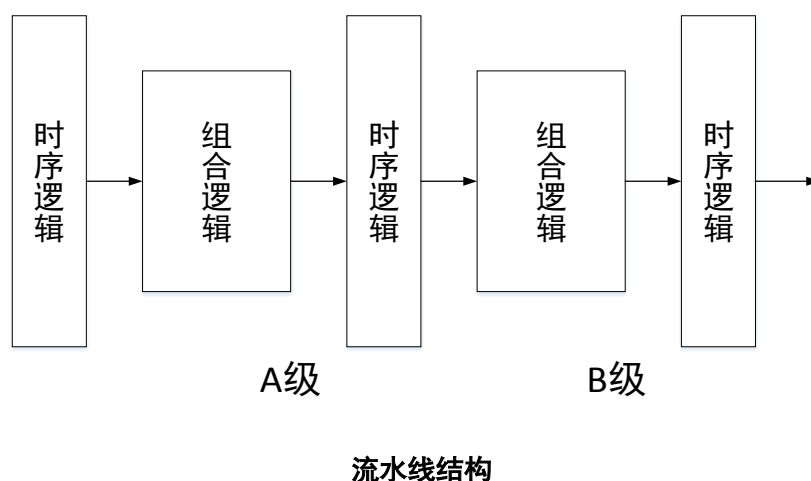
第一章 流水线基本概念

1.1 流水线基本原理

首先这里介绍的流水线结构只是一种流水线的实现。存在很多种其他的实现，同学们可以自己查阅资料学习。

在本指导书的设计中，流水线的每级都分成两个部分：**组合逻辑**和**时序逻辑**部分（当然这种划分不是严格的，例如 IF 和 WB 就很特殊），组合逻辑的输出传递到时序逻辑的输入。每一级的组合逻辑接受上一级的时序逻辑的输出，同时组合逻辑的输出传递到本级的时序逻辑部分。

时序逻辑的部分构造相当简单：每当**时钟上升沿**来临时，它就把输入刷新到对应的输出，这种刷新导致了后面组合逻辑部分的变化；时钟信号保持时，组合逻辑部分计算得到输出并维持电平；下一个时钟上升沿来临时，这一级的时序逻辑把输出刷新到下一级的组合逻辑部分。简单来说，组合逻辑**接受和产生控制信号**，时序逻辑负责**信号的传递**。



1.2 五级流水线结构

① IF：取指

取值阶段，IF 从 ROM 拿到指令输出到它的时序逻辑并保持，同时 $PC+=4$ 。在处理分支指令时，PC 的变化还会受其他信号的控制，这些信号一般是从 ID 阶段发出的。

② ID: 译码

译码阶段产生后续部件需要用到的控制信号和数据,在这个阶段读取寄存器的值并输出保持。另外,对于分支指令,也要在这一阶段设置跳转的地址。ID 阶段是整个流水线中最为重要的部件,它控制了后续几个部件的工作行为。

③ EX: 执行

执行阶段的核心部件是 ALU。它根据 ID 阶段的信号执行对应的运算,把结果输出。大部分的运算都可以使用组合逻辑瞬间完成,少部分的运算如除法等需要用到多个周期。多周期的运算需要用到**流水线暂停机制**,在后文我们会介绍这种暂停机制。

④ MEM: 访存

访存阶段从输入的控制信号中拿到要写入/读取的内存地址。大部分指令在这里都没有操作,所以大部分情况下访存阶段的组合逻辑只是把输入传递到输出。

⑤ WB: 写回

执行到这一阶段的指令已经拿到了结果。WB 阶段的主要功能是把结果写入寄存器。

第二章 流水线实现 1

本章实现一个具有基本功能的 CPU。

2.0 准备工作

下面是开始编写之前的准备工作, **请同学们认真阅读**。

- ① 在我们的 CPU 中涉及到多个具有固定宽度的总线,传递指令、传递数据、传递寄存器编号等都需要用到某种宽度的总线。我们把这些总线的宽度写成宏定义的形式,在后期如果发现某个总线不够用时,可以很方便地扩大其宽度,否则就要在庞大的代码库中寻找哪里应用了这个总线。总线宽度的定义我们放在 **global.v** 文件中。它的内容类似下面的形式:

global.v 的样例

```

1  // 地址线宽度为 32
2  `define AddrBus          31:0
3  `define AddrBusWidth    32
4  // 指令长度为 32
5  `define InstBus         31:0
6  `define InstBusWidth    32
7  // 数据总线宽度为 32
8  `define DataBus         31:0
9  `define DataBusWidth    32
10 // 两倍的数据总线宽度
11 `define DoubleDataBus    63:0
12 `define DoubleDataBusWidth 64

```

- ② 在 MIPS 指令集中，操作码、FUNCT 等都是人为定义的。把这些“magic code”写成 Verilog 宏定义的形式有助于提高代码的可读性。因此我们把这些定义分类放在多个文件里。在后面的开发过程中我们会逐步添加这些文件的内容：

定义文件的列表

文件名	用途
op_def.v	定义 OP 字段内容
funct_def.v	定义 SPECIAL 指令中 FUNCT 字段的内容
regimm_def.v	定义 REGIMM 类指令中 rt 字段的内容
cop0_def.v	定义 COP0 操作指令中相关内容

下面是 op_def.v 的一部分：

op_def.v 样例

```

1  `define OP_SPECIAL      6'b000000

```

```

2  `define OP_REGIMM      6'b000001
3  `define OP_J           6'b000010
4  `define OP_JAL         6'b000011
5  `define OP_BEQ         6'b000100
6  `define OP_BNE         6'b000101
7  `define OP_BLEZ        6'b000110
8  `define OP_BGTZ        6'b000111

```

- ③ 注意到在五级流水线中存在 4 个时序逻辑的部分：IF_ID、ID_EX、EX_MEM、MEM_WB。这些部分的工作只是在时钟上升沿时，把输入传递给输出。显然它们的功能是高度一致的，只有宽度上的区别。这里在实现时可以考虑使用 Verilog 的**带参数模块**——它能够显著降低代码量，因而可能出错的地方也就变少了。这里的实现比较简单，同学们可以自己实现。
- ④ 流水线 CPU 模块很多，模块之间连线复杂。如果还使用 Verilog 去编写顶层模块，毫无疑问工作量会很高并且容易出错。这里我们可以用到 Vivado 的 Design Block 功能：用户只需要绘制模块之间的连线，Vivado 会自动生成对应的模块。如何使用这一功能请参见附录部分。

接下来我们创建顶层模块 SoC，它只有两个输入 clk 和 rst，分别是时钟信号和重置信号。然后创建一个 Design Block，这里用来放置流水线的各个模块。最后我们在 SoC 中引用它即可。在这里 SoC 实际上是包装了 Design Block，因为 Vivado 似乎并不支持 Design Block 作为顶层模块使用。下面是 SoC 的内容和新建的 Design Block 的图像：

SoC.v

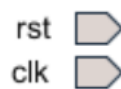
```

1  `timescale 1ns / 1ps
2
3  module SoC(
4      input clk,

```

```
5    input rst
6    );
7    SoC_design soc(.clk(clk), .rst(rst));
8    endmodule
```

SoC_design(Block Design)



（因为咱什么都没做对吧先放着……）

开始之前我们先看一下 CPU 的主要模块：

- ① ROM/RAM/RegFile：指令存储器、数据存储器和寄存器；
- ② PC：IF 阶段的核心部件；
- ③ ID/EX/MEM：译码、执行、访存阶段的核心部件（IF 阶段就是 PC，WB 阶段没有组合逻辑，只连接到寄存器）；
- ④ IF_ID/ID_EX/EX_MEM/MEM_WB：每两个阶段之间的时序逻辑部分。

2.1 外围模块

这些模块与流水线本身的结构没有太大关系，因此我们单独抽出这些部分。实现这些模块之后基本不需要做修改，在顶层模块中引用即可。

2.1.1 ROM

ROM 部分用于保存我们预先编译好的程序。CPU 上电运行后，从 ROM 的 0 号地址读取第一条指令执行。在这里为了简化，我们把 ROM 设计为异步的组合逻辑：给出一个地址，在 ROM 的输出端口就立即得到对应的指令。它的端口定义如下：

ROM 端口定义

名称	类型	宽度	方向	用途
en	wire	1	input	使能信号，为 1 时输出指令
addr	wire	32	input	指令地址
inst	wire	32	output	输出指令

ROM 比较简单，这里给出一个实现：

ROM.v

```
1  `timescale 1ns / 1ps
2  `include "global.v"
3  module ROM(
4      // 使能信号，输入为 1 时表示 Enable，为 0 时表示 Disable
5      input en,
6      // 输入地址
7      input[`AddrBus] addr,
8      // 输出地址
9      output[`InstBus] inst
10 );
11 // 写成 8 位一组是因为
12 // objcopy 的 verilog 输出格式是这样
13 // 512=2^9, 512B 大小, 2^7=128 条指令
14 // 但是对外则"假装"有完整的 2^32B=4GB 空间
15 reg[7:0] rom[0:511];
16 // 从文件读取"固件"
17 initial $readmemh("C:/cpu_v3/rom_src/inst_rom.bin", rom);
18 // MIPS 是大端，低地址存放高位
19 // 这里 add 最后两位实际是不需要的（4 字节对齐）
```

```

20    // 但是保留它可以暴露潜在的 bug
21    assign inst[ 7: 0] = (en ? rom[addr + 3] : 8'b0);
22    assign inst[15: 8] = (en ? rom[addr + 2] : 8'b0);
23    assign inst[23:16] = (en ? rom[addr + 1] : 8'b0);
24    assign inst[31:24] = (en ? rom[addr + 0] : 8'b0);
25 endmodule

```

有些地方需要注意：

- ① 在这个文件的开头我们引入了“global.v”这个文件，它的作用和 C 语言的 `#define` 类似：声明一个宏，在编译时做替换。在这里我们把地址线和指令长度的宽度变成宏定义。
- ② 注意到上面的 ROM 实现中，存储器的最小单元是 1 字节而不是 4 字节。这种设计是考虑到 GCC 的编译输出格式，它就是以 1 字节为最小单元。为了能够用上 GCC 为我们编译汇编程序，就把 ROM 的最小单元也变成 1 字节。
- ③ `readmemh` 函数中文件路径建议使用**绝对路径**。注意这里的文件路径和 C 语言的语法是类似的，反斜杠“\”表示转义字符。所有出现“\”的地方都要写成“\\”或者“/”。

2.1.2 RAM

RAM 用于保存数据。RAM 的设计更复杂一些：读取是异步的，给出地址就得到数据；写入是同步的，只有当时钟上边沿来临时才能做写入。因此 RAM 的接口更多：

RAM 端口定义

名称	类型	宽度	方向	用途
clk	wire	1	input	时钟信号
rst	wire	1	input	重置信号
enable	wire	1	input	整体的使能信号

write_en	wire	1	input	写使能信号，
write_sel	wire	4	input	要写入哪些字节的标志
addr	wire	32	input	输入地址，严格按照 4 字节对齐
data_in	wire	32	input	要写入的数据
data_out	reg	32	output	读出的数据，传递到 CPU

write_sel 信号比较复杂：我们的写入数据是 32 位的，但是有时候我们只希望写入到某一个字节，这时候就需要 write_sel 来控制写入哪些字节到 RAM。当 write_sel=4'b1111 时，32 位的输入都被写入；为 4'b0011 时，写入低 16 位；为 4'b0100 时，只写入输入的 23~16 位；依此类推。

2.1.3 RegFile

RegFile 是 MIPS 汇编中使用到的 32 个通用寄存器。考虑到很多 MIPS 指令都有两个寄存器参数，我们给 RegFile 设计两个读端口和一个写端口。寄存器的设计和单周期 CPU 的类似，读数据是异步的，给出寄存器编号就得到输出；写数据是同步的，只有时钟边沿来临时才能写入数据。其中，译码（ID）阶段需要读数据的功能，写回（WB）阶段需要写数据的功能。它的接口定义如下：

RegFile 的接口定义

名称	类型	宽度	方向	用途
clk	wire	1	input	时钟信号
rst	wire	1	input	重置信号
read_en_1	wire	1	input	读使能信号 1
read_addr_1	wire	5	input	要读取的寄存器编号 1
read_data_1	reg	32	output	输出数据 1
read_en_2	wire	1	input	读使能信号 2
read_addr_2	wire	5	input	要读取的寄存器编号 2
read_data_2	reg	32	output	输出数据 2

write_en	wire	1	input	写使能信号
write_addr	wire	5	input	要写入的寄存器编号
write_data	wire	32	input	要写入的数据

寄存器的部分实现比较简单，同学们自行实现即可。**注意 0 号寄存器是不允许写入的。**

2.2 取指阶段 IF

取指阶段的核心部件是 PC。除了 PC 之外就只剩下 IF 与 ID 之间的时序逻辑部分。PC 的结构很简单，它的主要功能是把指令地址传递给 ROM。由于现在我们还不需要实现跳转指令所以 PC 的定义只有 4 个：

PC 模块接口定义

名称	类型	宽度	方向	用途
clk	wire	1	input	时钟信号
rst	wire	1	input	重置信号
rom_en	reg	1	output	给 ROM 的使能信号
addr	reg	32	output	给 ROM 的指令地址

PC 的逻辑如下：

- rom_en 与 rst 信号相关，每当时钟上升沿来临时，如果重置信号为 1，那么 rom_en 就为 0
- 时钟上升沿来临时，如果 rom_en 是 1，那么把 addr 加上 4。（注意这里是判断 rom_en 信号，这样刚刚上电时就能从 0 号地址开始）

实现代码如下：

PC.v

```

1 `timescale 1ns / 1ps
2 `include "global.v"
3

```

```

4  module PC(
5      input clk,
6      input rst,
7
8      output reg rom_en,
9      output reg[`AddrBus] addr
10 );
11
12     always@(posedge clk) begin
13         rom_en <= !rst;
14     end
15
16     always@(posedge clk) begin
17         // 这里不是判断 rst, 而是 rom_en
18         if (!rom_en)
19             addr <= `AddrBusWidth'b0;
20         else
21             addr <= addr + 4;
22     end
23 endmodule

```

接下来是 IF 和 ID 之间的时序逻辑部件。它的输入是 PC 输出的地址和来自于 ROM 的指令数据。每个时钟上升沿的时候 IF_ID 把输入刷新到输出。有了 components.v 里面的模块，IF_ID 可以写成下面的形式：

IF_ID.v

```

1  `timescale 1ns / 1ps
2  `include "global.v"

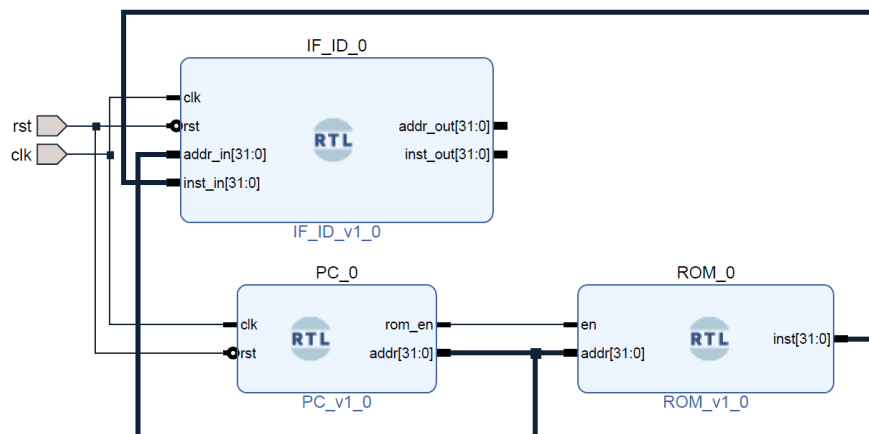
```

```

3
4  module IF_ID(
5      input clk,
6      input rst,
7
8      input[`AddrBus] addr_i,
9      input[`InstBus] inst_i,
10
11     output [`AddrBus] addr_o,
12     output [`InstBus] inst_o
13 );
14
15     PipelineDeliver#(`AddrBusWidth) ff_addr(
16         clk, rst, addr_i, addr_o);
17     PipelineDeliver#(`InstBusWidth) ff_inst(
18         clk, rst, inst_i, inst_o);
19 endmodule

```

PipelineDeliver 是作者自己实现的 D 触发器，它的代码很简单这里就不给了。实现它们之后我们把已经完成的部件加到 SoC 的设计图上并连线：



IF 阶段的作用就是输出指令和对应的地址。

2.3 译码阶段 ID

译码阶段的主要工作是：解析从 IF 传递过来的指令内容；根据需要从 RegFile 中拿到对应寄存器的值；生成后续阶段（EX、MEM、WB）需要的控制信号。因而 ID 阶段有 3 个部件：ID 本身的组合逻辑（ID.v）、寄存器 RegFile 和时序逻辑 ID_EX。RegFile 在前面已经实现，而 ID_EX 只要使用就 PipelineDeliver 可以完成，因而重点是梳理 ID 模块的输入输出。

- ① 首先 ID 模块需要支持 rst 信号，所以有一个输入是 rst；
- ② 显然 ID 模块需要从 IF 拿到指令内容和此时的 PC 值（在跳转指令时会用到），因而有两个输入：inst 和 addr；
- ③ ID 模块还需要从寄存器读取数据，所以寄存器模块的两个读写端口也要连接到 ID 模块上；
- ④ 指挥 EX 阶段需要指定 ALU 的输入，它们分别是：ALU 的操作码 FUNCT、移位指令的参数 SHAMT、ALU 的两个操作数 1 和 2；
- ⑤ 控制 EX 阶段计算得到结果后，应该还需要把它写入到寄存器，因而还有控制 WB 阶段的写入使能信号 write_reg_en 和对应的寄存器编号 write_reg_addr；
- ⑥ ID 是整个流水线的核心部件，因而应该还存在控制 PC、控制 MEM、控制 CP0 的信号。但是目前我们还用不着控制这两个部分。

综上，我们可以得到一张 ID 模块的输入输出表：

标题样例

名称	类型	宽度	方向	用途
rst	wire	1	input	重置信号
addr	wire	32	input	从 IF 拿到的指令地址
inst	wire	32	input	从 IF 拿到的指令内容
reg_data_1	wire	32	input	从 RegFile 拿到的数据 1
reg_data_2	wire	32	input	从 RegFile 拿到的数据 2
reg_read_en_1	reg	1	output	给 RegFile 的使能信号 1

reg_read_en_2	reg	1	output	给 RegFile 的使能信号 2
reg_addr_1	reg	5	output	给 RegFile 的寄存器地址 1
reg_addr_2	reg	5	output	给 RegFile 的寄存器地址 2
funct	reg	6	output	给 EX 的操作码
operand_1	reg	32	output	给 EX 的操作数 1
operand_2	reg	32	output	给 EX 的操作数 2
shamt	reg	5	output	给 EX 的移位指令参数
write_reg_en	reg	1	output	给 WB 的写寄存器信号
write_reg_addr	reg	5	output	给 WB 的写寄存器地址

注意到最后两个信号是给 WB 的，但是因为流水线本身的结构，这两个信号需要先传递给 EX 部件，然后传递给 MEM 部件，最后才可以被 WB 部件接受。

确定接口后我们来实现 ID 的内容。首先我们以 ORI 指令和 OR 指令为例：

ORI 指令的 OP 是 2'b001101，我们在 op_def.v 中添加 OP_ORI 这个宏。OR 指令的 OP 是 2'b000000，查询 MIPS 手册我们知道这个操作码表示 SPECIAL，所以我们在 op_def.v 中也添加 OP_SPECIAL 这一行。以 OP_SPECIAL 为开头的指令有很多，它们通过 5~0 位的 FUNCT 段相互区分。

这里我们做一个归一化的处理。查询 MIPS 手册我们得知，OR 和 ORI 除了操作数 2 的来源不一样（一个来自于立即数的无符号扩展，一个来自于寄存器），要写入的寄存器地址不一样（一个是 rd，一个是 rt），在 ALU 看来它们其实是一种运算。注意到前面 ID 模块添加了给 WB 模块的写入寄存器地址和相关的使能信号，那么写入寄存器地址的区别就可以通过这部分来解决。而操作数来源的不同更好解决：只要控制向 EX 模块输出的 operand_1 和 operand_2 的内容就行了。这样，在 EX 模块看来，它只需要做一个 OR 操作，至于这个 OR 操作是来自于 ORI 指令还是 OR 指令，EX 模块并不需要关注。

同样能做这种归一化处理的指令还有很多，例如 addi 与 add, addiu 与 addu，

andi 与 and 等等。当然，这只是 ID 和 EX 模块的一种实现思路。不使用这种归一化处理也是可以的，无非是让 EX 的控制信息更加详细一些。如果你觉得这样的设计与你自己的想法不同，可以使用自己思路。当然，思路的调整也会带来接口的变化。

接下来我们就付诸实现。首先我们要完成对寄存器部分的操作。对于 OP_ORI 来说，它只需要读取寄存器 rs 的值；对于 OP_SPECIAL 来说，它需要读取寄存器 rs 和 rt 的内容，所以有下面的代码：

ID 模块：处理 RegFile 的信号

```
1  wire[`InstOpBus] inst_op          = inst[31:26];
2  wire[`RegAddrBus] inst_rs         = inst[25:21];
3  wire[`RegAddrBus] inst_rt         = inst[20:16];
4
5  always@(*) begin
6      if (rst) begin
7          reg_read_en_1 <= 0;
8          reg_read_en_2 <= 0;
9          reg_addr_1 <= 0;
10         reg_addr_2 <= 0;
11     end
12     else case(inst_op)
13         `OP_ORI: begin
14             reg_read_en_1 <= 1;
15             reg_read_en_2 <= 0;
16             reg_addr_1 <= inst_rs;
17             reg_addr_2 <= 0;
18         end
19         `OP_SPECIAL: begin
```

```

20         reg_read_en_1 <= 1;
21         reg_read_en_2 <= 1;
22         reg_addr_1 <= inst_rs;
23         reg_addr_2 <= inst_rt;
24     end
25     default: begin
26         // 默认情况下不读寄存器
27         reg_read_en_1 <= 0;
28         reg_read_en_2 <= 0;
29         reg_addr_1 <= 0;
30         reg_addr_2 <= 0;
31     end
32 endcase
33 end

```

（写 case 语句的时候总是加上 default 是个好习惯）

然后是 **funct**。通过前面的归一化处理我们可以有下面的代码（记得把 OR 指令的 FUNCT 值写入到 funct_def.v，也就是`define FUNCT_OR 6'b100101）：

ID 模块：处理 EX 的操作码

```

1  wire[`FunctBus] inst_funcnt           = inst[5:0];
2  always@(*) case(inst_op)
3      `OP_SPECIAL: funct <= inst_funcnt;
4      `OP_ORI: funct <= `FUNCT_OR;
5      // FUNCT_NOP 的取值是 2'b111111
6      default: funct <= `FUNCT_NOP;
7  endcase

```

FUNCT_NOP 不是 MIPS 手册里的，只是随意选取一个 MIPS 没有利用到的值（比如 2'b1111111）代替一下，表示不做任何事情。更严格的情况下，应该添加一个输出 flag 表示该指令是否需要 EX 中的 ALU 参与。

接下来是给 EX 阶段的 **operand_1** 和 **operand_2**（操作数 1 和操作数 2）。查阅 MIPS 手册，我们得知 OR 指令的操作数 1 是 rs 对应的寄存器值，操作数 2 是 rt 对应的寄存器值；对于 ORI 指令，操作数 1 是 rs 对应的寄存器值，操作数 2 是立即数（15~0 位）的无符号扩展。由于前面我们已经给出了寄存器读取的信号，而寄存器读取是异步的，所以在此时 reg_data_1 和 reg_data_2 的信号值就是对应寄存器的值。代码实现如下：

ID 模块：处理 EX 的操作数

```
1  // handle operand_1
2  always@(*) begin
3      if(rst) operand_1 <= 0;
4      else case(inst_op)
5          `OP_ORI: operand_1 <= reg_data_1;
6          `OP_SPECIAL: operand_1 <= reg_data_1;
7          default: operand_1 <= 0;
8      endcase
9  end
10 // handle operand_2
11 wire[15:0] inst_imm          = inst[15: 0];
12 wire[31:0] zero_extended_imm = {16'b0, inst_imm};
13 always@(*) begin
14     if(rst) operand_2 <= 0;
15     else case(inst_op)
16         `OP_ORI: operand_2 <= zero_extended_imm;
```

```
17      `OP_SPECIAL: operand_2 <= reg_data_2;
18      default: operand_2 <= 0;
19  endcase
20 end
```

注意 ORI 是无符号扩展（zero-extended）。

然后是 shamt 输出。这部分我们暂时没有用到，直接把 SHAMT 字段赋给输出就行了。保险起见还可以加上一个判断，只有当指令是 OP_SPECIAL 时才输出 shamt，否则输出 0。

然后是给 WB 的写寄存器信号与地址。对于 OR 指令，写入地址是 rd；对于 ORI 指令，写入地址是 rt。这部分的写法和前面是类似的，就不贴出代码了。

最后是 ID_EX 模块。ID 模块要传送给 EX 模块的信号有 funct、operand_1、operand_2、shamt、write_reg_en 和 write_reg_addr。和 IF_ID 一样编写即可。最后我们把这两个模块添加到 SoC 中去，得到的图可以查看 PDF。

2.4 执行阶段 EX

目前 EX 阶段的主要工作是执行运算并传出地址。在 ID 阶段我们已经给出了操作码、操作数以及写入寄存器的信号和地址。也就是说，结果放到哪里我们已经在 ID 阶段指定了，EX 要做的就是计算出结果。代码如下：

标题样例

```
1  `timescale 1ns / 1ps
2  `include "global.v"
3  `include "funct_def.v"
4
5  module EX(
6      input rst,
7      input[`FunctBus] funct,
```

```
8      input[`DataBus] operand_1,
9      input[`DataBus] operand_2,
10     input[`ShamtBus] shamt,
11     input write_reg_en_in,
12     input[`RegAddrBus] write_reg_addr_in,
13
14     output[`DataBus] result_out,
15     output write_reg_en_out,
16     output[`RegAddrBus] write_reg_addr_out
17 );
18 // 这两个信号直接传递出去
19 assign write_reg_addr_out = (rst ? 0 : write_reg_addr_in);
20 assign write_reg_en_out = (rst ? 0 : write_reg_en_in);
21 reg[`DataBus] result;
22 assign result_out = (rst ? 0 : result);
23
24 always@(*) begin
25     case (funct)
26         `FUNCT_OR: result <= (operand_1 | operand_2);
27         default: begin
28             result <= 0;
29         end
30     endcase
31 end
32 endmodule
```

同样, EX_MEM 的输入和输出也是对应的(结果、是否写入寄存器、寄存器编号)。

把 EX 和 EX_MEM 模块加入到 SoC 之后的图可以看附件的 PDF。

2.5 访存阶段 MEM

因为我们暂时没有实现访存阶段的指令，所以这里的组合逻辑部分只是简单地把输入传递到输出即可。对应的 MEM_WB 也是和其他的时序逻辑模块一样。

2.6 写回阶段 WB

最后一部分 WB 没有组合逻辑，只需要把 MEM_WB 的三根信号接入 RegFile 即可（直接在 Diagram 中操作）。

完成后的基本 CPU 电路图在附件中可以看到。

2.7 解决数据相关问题

目前我们的五级流水线 CPU 已经初步完成，但是依然存在数据相关的问题。在 CPU 中数据相关有三种情况（以前后执行的两条指令 A 和 B 为例，先执行 A，然后执行 B）：

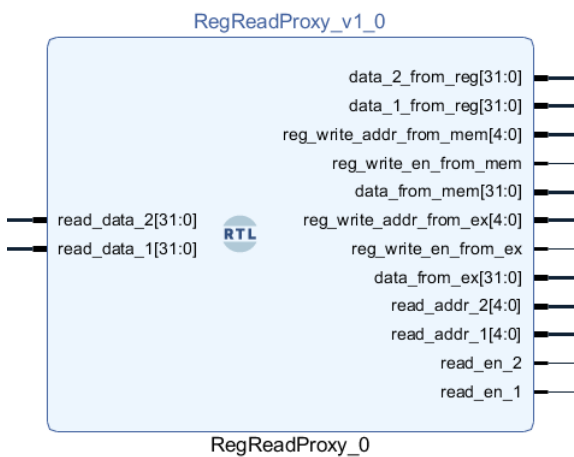
- ① Read After Write, RAW。A 写入寄存器，B 读取寄存器。由于流水线的特点，有可能出现 A 写入之前 B 就尝试读取同一个寄存器，显然 B 得到的值是不对的；
- ② Write After Read, WAR。A 读取寄存器，B 写入寄存器。如果在 A 读取之前，B 就把结果写入同一个寄存器，那么 A 指令拿到的数据就是不对的；
- ③ Write After Write, WAW。A 和 B 都写入同一个寄存器，AB 都执行完之后这个寄存器的值应该是 B 写入的值。如果 A 在 B 之后写入寄存器，就会导致寄存器的值不正确。

从我们已经实现的内容分析，由于我们的流水线的 WB 阶段是逐条进行的，所以不存在 WAW 冲突；另外，由于读寄存器在 ID 阶段，写寄存器在 WB 阶段，所以也不存在 WAR 冲突。

RAW 冲突是有可能发生的。从 EX 得到的结果，要经过 MEM、WB 两个周期才会写入到寄存器。在这期间任何指令尝试读取这个寄存器的值都会拿到错误的结果。RAW 冲突不是不可能解决的，因为每条指令在一个流水线部件上都是依次执行，读寄存器的指令在 ID 阶段时，前一条指令在 EX 阶段已经得到了结果。

在这里我们使用**数据前推**来解决 RAW 冲突，也就是使用一个额外的部件把计算结果提前推送给 ID 模块。

我们创建一个新的模块 **RegReadProxy**，它在 **RegFile** 的读端口上做了一层代理。从 ID 模块来看，它和实际的 **RegFile** 没有区别。但是从它读取得到的寄存器值都是经过前推的、正确的。需要被前推的数据应当是从 **EX** 模块、**MEM** 模块的输出端口连接到 **RegReadProxy**：



左侧是输出端口，这两个接口连接到 ID 的 **read_data_1** 和 **read_data_2**。右侧是输入端口，其中来自于 **RegFile** 的数据输入两个（最上方），然后是来自于 **MEM** 模块的输出 3 个，**EX** 模块的输出 3 个，来自于 ID 模块的寄存器读信号和地址 4 个。**RegReadProxy** 的设计很简单：如果要读取的数据正好是 **MEM** 或者 **EX** 模块要写入的，那么传送给 ID 模块的是要写入的数据；否则给把 **RegFile** 的输出给 ID。注意我们需要先判断 **EX** 再判断 **MEM**，因为 **EX** 的输出是 **MEM** 输出的前一条指令。

加入 **RegReadProxy** 之后的电路图见 PDF 附件。

2.8 实现其他指令的提示

完成这一部分之后我们的流水线实际上已经能支持扩展实现大部分的算术指令，包括：

ADD	ADDI	ADDU	ADDIU		
SUB	SUBU	SLT	SLTI	SLTU	SLTIU

AND	ANDI	LUI	NOR	OR	ORI	XOR	XORI
SLL	SLLV	SRA	SRAV	SRL	SRLV		

实现这些指令需要注意下面几点：

- 指令的操作数是什么，是否需要读取寄存器
- 如果是 `OP_SPECIAL` 指令，还需要注意 ID 给出的信号是否符合指令的要求
- 指令的写回寄存器地址是什么
- 宏定义文件的部分也要加上这些指令的内容

第三章 流水线实现 2

这一章我们继续完善 CPU 的功能，主要是实现乘法指令和除法指令需要的 HILO 寄存器和相关指令。

3.1 实现 HILO 寄存器

HILO 寄存器用来保存乘法或者除法指令的结果，它有两个 32 位寄存器 HI 和 LO。和 RegFile 一样，它的读也是异步的，写则是同步的。由于只有两个寄存器所以我们直接把 reg 在暴露 output 部分：

HILO.v

```
1  `timescale 1ns / 1ps
2
3  `include "global.v"
4
5  module HILO(
6      input clk,
7      input rst,
8      input write_en,
9      input[`DataBus] hi_in,
10     input[`DataBus] lo_in,
11
12     output reg[`DataBus] hi,
13     output reg[`DataBus] lo
14 );
15
16     always@(posedge clk) begin
```

```
17         if (rst) begin
18             hi <= 0;
19             lo <= 0;
20         end
21     else if (write_en) begin
22         hi <= hi_in;
23         lo <= lo_in;
24     end
25 end
26 endmodule
```

直接读写 HILO 的指令有 MFHI、MFLO、MTHI、MTLO。查阅手册我们知道他们都是 OP_SPECIAL 类指令，依靠 FUNCT 字段做区分。在前面的设计中我们有一个归一化处理：如果是 OP_SPECIAL 指令，那么操作数 1 是 rs，操作数 2 是 rt，写寄存器信号为有效，地址是 rd，给 EX 的 FUNCT 就说指令中的 FUNCT。

这种归一化处理在应对上述 4 条指令的时候是否有效呢？我们下面进行分析：

- ① MFHI 指令的 rs 是 0，rt 也是 0，rd 是要写入的寄存器编号。在处理 MFHI 时，ID 模块没有查看 FUNCT 字段而是传递给 EX。EX 从 FUNCT 知道要执行 MFHI，此时从 HILO 寄存器拿到 HI 的值作为结果，交给下一级，没有问题；
- ② MFLO 和 MFHI 一样的处理方式；
- ③ MTHI 和 MFLO 指令的 rs 是数据源的寄存器编号，rt 和 rd 是 0。EX 从 FUNCT 得知要把 rs 写到 HI 或者 LO。注意到 ID 模块处理 OP_SPECIAL 时把写寄存器信号设置为 1，但是这里要“写”的是 0 号寄存器，所以这里不需要特殊的修改。

HILO 的值在 EX 模块要用到，同时 EX 模块还要输出：HILO 的写信号、要写入的 HI 或者 LO 值。注意到我们没有信号指定要写入 HI 还是 LO——写入 HI 时把原来的 LO 值也一并传递即可。这些信号一直会传递到 WB，所以我們也要给后续模

块加上对应的信号。

注意到 HILO 也是存在数据冲突的问题的 (RAW)，这里也是用数据前推，在 EX 模块获得 HILO 值的端口之前加一个代理 HILOReadProxy，由它确定 HILO 的值是多少。这个代理的输入来自于 MEM 模块和 WB 模块的输出。

3.2 实现跳转指令与延迟槽

延迟槽是 MIPS 指令集中比较特殊的一个特性：在执行到跳转指令之后，CPU 的执行流程并不立即跳转，而是在执行跳转指令之后的一条指令后，再进行跳转。在五级流水线中实现延迟槽是非常自然的：

- ① 跳转指令达到译码阶段时，才会向 IF 发出跳转的信号；
- ② 此时跳转指令的下一条指令已经被 IF 取出；
- ③ 时钟上边沿来临，跳转指令进入 EX 阶段，它的下一条指令进入 ID 被译码，延迟槽的行为就体现出来了。此时 PC 已经被更新，新地址的指令被 IF 取出，完成跳转。

因而为了实现跳转指令，我们只需要在 ID 部件上增加一些输出：控制是否跳转的信号 `branch_flag` 和要跳转到的新地址 `branch_addr`。这两个信号连接到 PC 上，所以我们还需要对 PC 做一些修改：

PC.v: 支持跳转

```
1  `timescale 1ns / 1ps
2  `include "global.v"
3
4  module PC(
5      input clk,
6      input rst,
7
8      input branch_flag,
9      input[`AddrBus] branch_addr,
```

```

10
11     output reg rom_en,
12     output reg[`AddrBus] addr
13 );
14
15     always@(posedge clk) begin
16         rom_en <= !rst;
17     end
18
19     always@(posedge clk) begin
20         // 这里不是判断 rst, 而是 rom_en
21         if (!rom_en)
22             addr <= `AddrBusWidth'b0;
23         else if(branch_flag)
24             addr <= branch_addr;
25         else
26             addr <= addr + 4;
27     end
28 endmodule

```

这样做的含义就是，如果 `branch_flag` 是 1，那么跳转到给定的值；否则就让 PC 加上 4。显然我们需要在 ID 部件中加入跳转地址生成的部分，这里以 BEQ 为例：

ID 模块：实现 BEQ

```

1  wire[`AddrBus] addr_plus_4 = addr + 32'h4;
2  wire[25:0] jump_addr = inst[25:0];
3  // 用于 beq/bne/bgtz/blez 的分支跳转

```

```
4  wire[`DataBus] imm_sll2_signedext = {{14{inst[15]}}, inst[15:0],
    2'b00};
5  always@(*) begin
6      if(rst) begin
7          branch_flag <= 1'b0;
8          branch_addr <= `AddrBusWidth'b0;
9      end
10     else case(inst_op)
11         `OP_BEQ: begin
12             if(reg_data_1 == reg_data_2) begin
13                 branch_flag <= 1'b1;
14                 branch_addr <= addr_plus_4 + imm_sll2_signedext;
15             end
16             else begin
17                 branch_flag <= 1'b0;
18                 branch_addr <= `AddrBusWidth'b0;
19             end
20         end
21         default: begin
22             branch_flag <= 1'b0;
23             branch_addr <= `AddrBusWidth'b0;
24         end
25     endcase
26 end
```

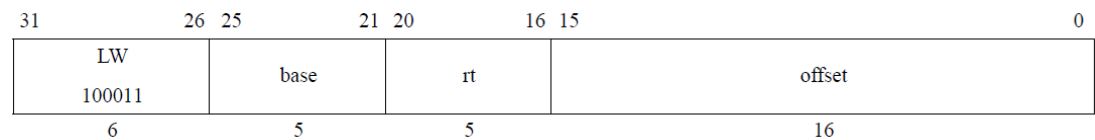
注意 BEQ 等指令都是对立即数的地址做有符号扩展然后左移两位，所以 BEQ 这类指令可以向前或者向后跳转。最后，在 SoC 的电路图中连接好 ID 和 PC 的接口即可。

其他类型的指令都和这里的操作类似，带 link 功能的指令还需要同时指定写入寄存器的地址。

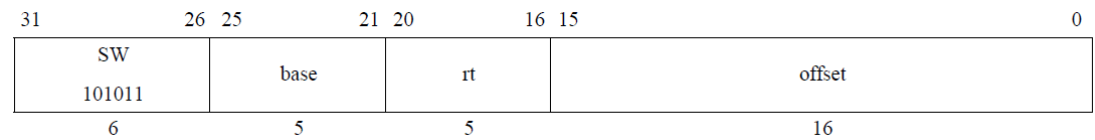
3.3 实现访存指令

到现在我们还没有用到 RAM。在 MIPS 中访存指令分为读取（load）和写入（save）两类。其中。load 指令有 lb、lbu、lh、lhu、lw；save 指令有 sb、sh 和 sw。这一节我们以 lw 和 sw 为例实现访存指令。

首先我们来看 lw 指令的结构：lw 是 I 型指令，OP 段是 2'b100011，rs 段是基地址所在的寄存器编号，rt 是写入寄存器的编号，imm 段是 16 位的偏移量。实际的内存地址由“寄存器值+有符号扩展后的偏移量”构成。从内存读取到数值后，写入到 rt 代表的寄存器。



sw 指令的结构也是类似的：sw 也是 I 型指令，OP 段是 2'b101011，rs 段和 imm 段共同构成要写入的内存地址。写入的数据来自于 rt 段代表的寄存器。和 lw 一样它的 imm 段也是进行有符号的扩展：



下面是实现思路。考虑到我们已有的结构，需要对 ID、EX 和 MEM 模块做一些修改：

- 对于 ID 模块，需要增加下列信号：mem_read_flag，指示是否是读内存；mem_write_flag，指示是否是写内存；mem_addr，表示需要访问的内存地址；mem_write_data，表示要写入的数据。处理 lw 时，把写寄存器的信号设置为 1，同时设置好对应的地址。处理这两条指令时，EX 模块不需要进行操作，所以给 EX 模块的 funct 是 FUNCT_NOP；
- 对于 EX 模块，处理 FUNCT_NOP 时不需要进行操作，只需要把上面新增的信号传递给 MEM 模块；
- 对于 MEM 模块，接收到上述信号之后完成读/写操作。写操作有地址和数据，

没有问题；读操作得到的结果需要覆盖掉从 EX 传递来的 result (EX 处理 NOP 时传出的 result 是 0)，交给 WB 阶段写入到寄存器。

第四章 进阶主题

4.1 流水线暂停机制

有些指令需要占用多个时钟周期才能完成。这其中最具代表性的就是除法指令 `div`。实现除法器有两种思路：一是单周期完成，需要用到 `for` 语句。在 Verilog 语句中 `for` 语句在编译时会被展开，所以单周期的除法器会被展开成一个具有多层的组合逻辑，占用大量的 FPGA 资源，十分浪费。第二种思路就是占用多个周期来实现。占用的周期越多，每个周期需要使用的资源就越少。

但是在当前 CPU 的设计里，每个时钟上边沿来临时就会刷新组合逻辑部分的输入，我们的除法指令还没有执行完毕就被后面的指令覆盖了。所以，我们必须添加一个部件来控制时序逻辑部分的刷新。如果组合逻辑的部分输入没有更新，那么它内部的状态就能够一直保持，也就实现了流水线暂停的功能。

首先我们需要添加一个新的模块 `Control`，它从组合逻辑的部分接收暂停流水线的请求信号，然后发出信号暂停对应的时序逻辑部分。它的接口如下：

Control 模块接口

名称	类型	宽度	方向	用途
<code>rst</code>	<code>wire</code>	1	<code>input</code>	重置信号
<code>request_from_id</code>	<code>wire</code>	1	<code>input</code>	从 ID 模块发出的暂停请求
<code>request_from_ex</code>	<code>wire</code>	1	<code>input</code>	从 EX 模块发出的暂停请求
<code>pause_pc</code>	<code>wire</code>	1	<code>output</code>	暂停 PC 的信号
<code>pause_if</code>	<code>wire</code>	1	<code>output</code>	暂停 IF 的信号
<code>pause_id</code>	<code>wire</code>	1	<code>output</code>	暂停 ID 的信号
<code>pause_ex</code>	<code>wire</code>	1	<code>output</code>	暂停 EX 的信号
<code>pause_mem</code>	<code>wire</code>	1	<code>output</code>	暂停 MEM 的信号
<code>pause_wb</code>	<code>wire</code>	1	<code>output</code>	暂停 WB 的信号

暂时我们只用到了 ID 和 EX 模块的暂停请求。每个时序逻辑部件（例如 IF_ID）都接受两个暂停信号，比如 IF_ID 模块接受 IF 暂停信号和 ID 暂停信号，依此类推。

如果 ID 模块发出了暂停请求，则需要暂停 PC、IF、ID，后面的 EX、MEM、WB 三个模块不暂停；如果 EX 模块发出了暂停请求，则需要暂停 PC、IF、ID、EX，后面的 MEM、WB 两个模块不暂停。显然整个 Control 模块都是组合逻辑，比较简单，这里就不给出实现了。

下面就是把这些 pause 信号加入到具体部件。下面是增加 pause 信号之后的 PC:

PC.v（支持暂停）

```
1  `timescale 1ns / 1ps
2  `include "global.v"
3
4  module PC(
5      input clk,
6      input rst,
7      input stall_pc,
8
9      input is_branch,
10     input[`AddrBus] branch_addr,
11
12     output reg rom_en,
13     output reg[`AddrBus] addr
14 );
15
16     always@(posedge clk) begin
17         rom_en <= !rst;
```

```

18     end
19
20     always@(posedge clk) begin
21         // 这里不是判断 rst, 而是 rom_en
22         if (!rom_en)
23             addr <= `AddrBusWidth'b0;
24         else if (!stall_pc) begin
25             if(is_branch)
26                 addr <= branch_addr;
27             else
28                 addr <= addr + 4;
29         end
30     end
31 endmodule

```

余下的就是把 pause 信号加入到 IF_ID、ID_EX、EX_MEM、MEM_WB。还记得之前的 PipelineDeliver 吗，我们把它扩展到支持 pause:

PipelineDeliver.v

```

1  module PipelineDeliver
2      #(
3          parameter width = 1
4      )
5      (
6          input clk, input rst,
7          input pause_current_stage,
8          input pause_next_stage,
9          input[width-1: 0] value,

```

```
10     output reg[width-1: 0] out
11 );
12     always@(posedge clk) begin
13         if (rst)
14             out <= 0;
15         else if (pause_current_stage && !pause_next_stage)
16             out <= 0;
17         else if (!pause_current_stage)
18             out <= value;
19     end
20 endmodule
```

`current_stage` 表示“当前级”，`next_stage` 表示下一级。例如，对于 EX_MEM，EX 就是当前级，MEM 则是下一级，依次类推。其他的修改就是把 `pause` 信号加入到这些时序逻辑的模块。

4.2 解决访存指令的 load 相关

之前实现的访存指令实际上存在 load 相关的问题：load 指令在 MEM 阶段才能拿到结果放到 MEM 的输出，而 load 之后的一条指令 A 此时已经在 EX 阶段，显然如果 load 指令的写入寄存器和 A 指令请求的寄存器是同一个，则 A 指令拿到的数据是错误的。

通过数据前推是没有办法解决这个问题的：数据前推的前提是，ID 阶段要读取的数据已经被 EX 计算出来，显然 load 指令无法满足这个要求。解决 load 相关的办法是使用流水线暂停机制：如果 ID 模块检测到 load 相关的问题，则向 Control 模块发出请求暂停流水线（`request_from_id`）——ID 模块后面的 EX、MEM 和 WB 继续执行，而 ID 及之前的模块维持信号不变。暂停一个周期后，load 指令达到 WB 阶段，获取到的数据通过数据前推达到 ID 模块。这时候 load 相关就已经解决了。

4.3 实现除法指令

除法指令我们采用试商法实现。我们把除法器变成一个独立模块 `Divider.v`。它的实现参考后面的附录。在这里我们主要关注 EX 模块如何处理 DIV 和 DIVU 指令。

首先参考后面的 Divider 实现给 EX 模块增加对应的接口 (`is_signed`、`start`、`cancel`、`operand_1`、`operand_2`、`ready`、`result`)。由于除法指令要求暂停流水线，所以我们还要把 Control 模块的 `request_from_ex` 连接到 EX 模块。接下来逐个分析新增的输出信号：

- ① `is_signed` 用于标识执行的除法是有符号还是无符号的。这里通过判断 FUCNT 是 `FUNCT_DIV` 还是 `FUNCT_DIVU` 就可以得出；
- ② 除法器的 `operand_1` 和 `operand_2`、`start` 信号和 `request_from_ex` 信号可以一起处理。显然，`rst` 为 1 时它们都是 0；`rst` 为 0 时，如果除法器的 `ready` 信号（准备就绪）是 0，表示除法器还在进行运算。此时我们要保持输出，所以 `op1` 和 `op2` 是从 ID 得到的操作数 1 和 2，`start` 信号给 1，`request` 信号也给 1 表示暂停流水线；如果除法器的 `ready` 信号是 1，则表示除法器完成了运算，此时保险起见把 `op1` 和 `op2` 继续维持，而 `start` 信号变成 0，`request` 信号为 0，让流水线继续运转。其他情况下，四个信号都给 0 即可。

4.4 协处理器 CP0

在 MIPS 处理器中，协处理器扮演了很重要的角色。包括异常、中断和浮点数在内的许多功能都是由协处理器实现的。其中，第 0 个协处理器 CP0 就负责 MIPS 处理器的异常和中断功能。

4.5 异常和中断

未完待续。。

第五章 附录

5.1 使用 GCC 工具链编译生成固件

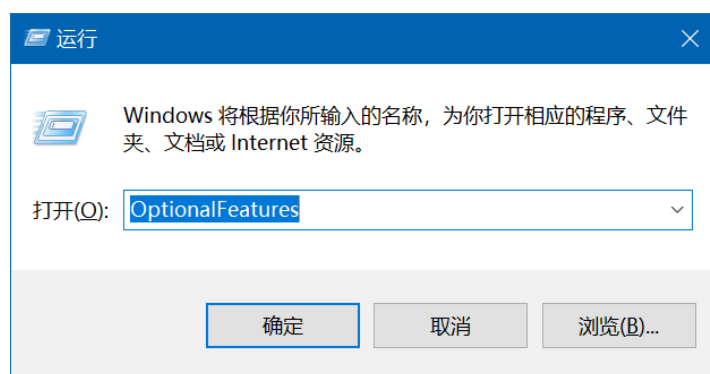
生成不同指令平台的可执行文件的过程称为**交叉编译**。在 Linux 上搭建交叉编译环境比较方便。这里以 Ubuntu 16.04 LTS 为例，讲解如何在 Linux 上生成 Verilog 支持的 ROM 初始化文件。

Ubuntu 操作系统可以在虚拟机中安装。下载 Ubuntu 16.04 LTS 或者更高版本的安装镜像，在 VMware 或者 Virtual Box 软件中安装。在 VMware 下安装完毕后，在虚拟机中安装 VMware Tools 可以实现拖拽文件到宿主机的操作。

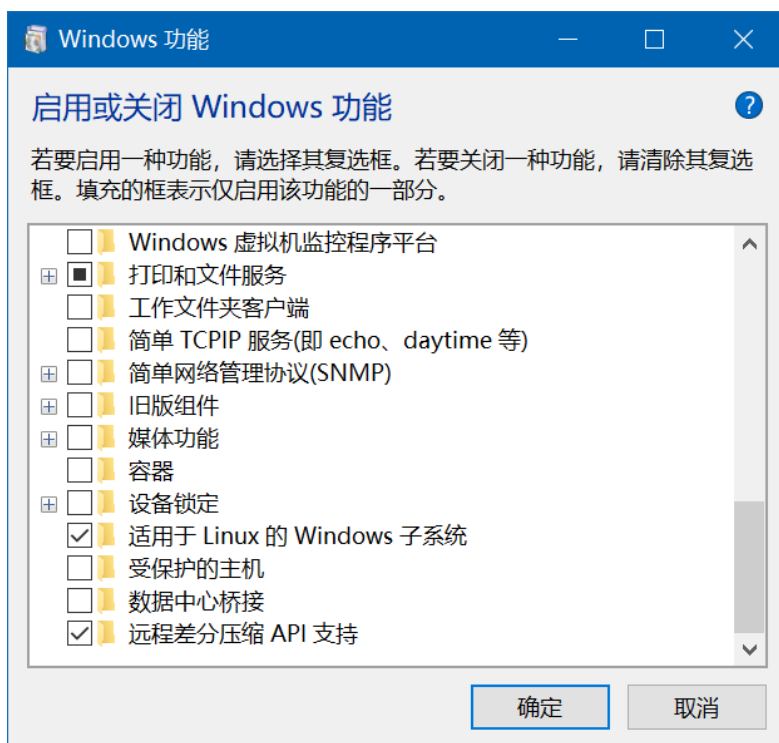
使用 Windows 10 Build 1607 及以上的同学还可以使用 **Windows Subsystem for Linux(WSL)**。这是由微软开发的一个工具，在 Windows 上模拟了大部分 Linux 的内核调用，使得我们能够在 Windows 环境下执行大部分的 Linux 命令。相比于虚拟机，这种方式更加简单。

5.1.1 WSL 的安装

1. 首先需要**启用 WSL 功能**。在 Windows 下按下 **Win+R** 组合键，输入“OptionalFeatures”，按下回车：



在弹出的窗口中找到“适用于 Linux 的 Windows 子系统”，勾选它，点击确定。



等待功能安装完毕。

2. 然后打开 Windows 10 的开发者模式。打开 Windows 10 的**设置**，搜索“开发者选项”，然后选择“开发人员模式”即可。

开发者选项

使用开发人员功能

这些设置只用于开发。

[了解更多信息](#)

☐ Microsoft Store 应用

仅安装 Microsoft Store 的应用。

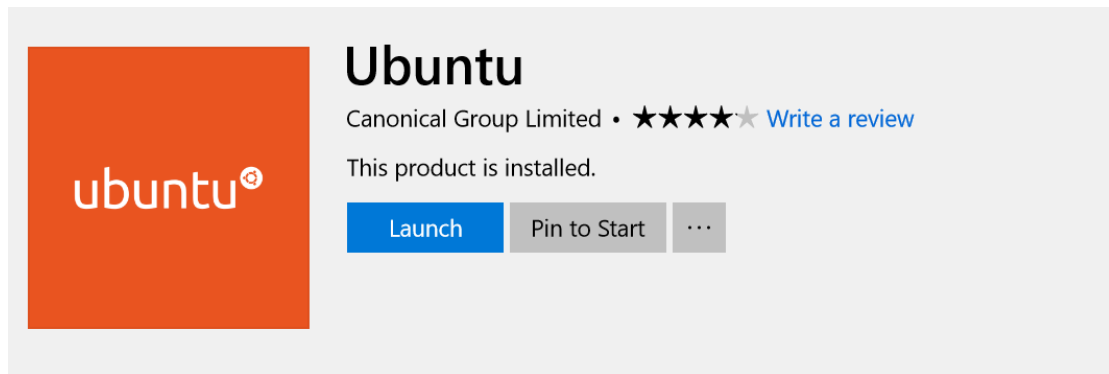
☐ 旁加载应用

从你信任的其他来源（例如工作区）安装应用。

☒ 开发人员模式

安装任何已签名的可信应用并使用高级开发功能。

3. 第三步对于不同的版本的 Windows 10 有所区别。对于较新的版本，如 Windows 10 Build 1709 及以上版本，打开 Microsoft Store，搜索“Ubuntu”，找到 Ubuntu 点击安装：



对于较旧的 Windows 10，如 Build 1607 版本，你需要打开 powershell (⊞+R 然后输入 “powershell”)，执行命令 bash：

```
Windows PowerShell
Windows PowerShell
版权所有 (C) 2016 Microsoft Corporation。保留所有权利。

PS C:\Users\qrato> bash
-- Beta 版功能 --
这将在 Windows 上安装由 Canonical 分发的 Ubuntu，
根据其条款的授权参见此链接：
https://aka.ms/uowterms

键入 “y” 继续：
```

两种方法都能够安装 WSL 下的 Ubuntu。由于系统需要下载 Ubuntu 的镜像，整个过程可能需要较长时间。安装完毕后，命令行会提示你输入 Linux 的用户名和密码，请牢记。

5.1.2 Vmware 下安装 Ubuntu

略。

5.1.3 安装交叉编译软件包

打开 Ubuntu 的命令行，输入下列命令：

```
sudo apt update
```

```
sudo apt upgrade
```

```
sudo apt install build-essential
```

```
sudo apt install binutils-mips-linux-gnu gcc-mips-linux-gnu
```

如果上面的命令下载速度太慢，执行“`sudo vi /etc/apt/source.list`”命令，修改软件源为中科大的镜像。在虚拟机下也是一样。

[illegible]

5.1.4 编写 Makefile

下面我们创建一个目录保存 makefile:

 $cd \sim$

```
mkdir rom_src
```

```
cd rom_src
```

vi Makefile

Makefile 的内容如下:

```
ifndef CROSS_COMPILE
    CROSS_COMPILE = mips-linux-gnu-
endif

CC = $(CROSS_COMPILE)as
LD = $(CROSS_COMPILE)ld

OBJCOPY = $(CROSS_COMPILE)objcopy
OBJDUMP = $(CROSS_COMPILE)objdump

OBJECTS = inst_rom.o
export CROSS_COMPILE

all: inst_rom.bin

%.o: %.S
    $(CC) -mips32 $< -o $@

inst_rom.om: ram.ld $(OBJECTS)
    $(LD) -T ram.ld $(OBJECTS) -o $@

inst_rom.bin: inst_rom.om
    $(OBJCOPY) --remove-section .reginfo \
    --remove-section .MIPS.abiflags -O verilog $< $@

clean:
    rm -f *.o *.om *.data *.bin
```

我们还需要创建 ram.ld 文件，输入 “vi ram.ld” 然后输入下面内容：

MEMORY

```
{  
    ram : ORIGIN = 0x00000000, LENGTH = 0x00010000  
}
```

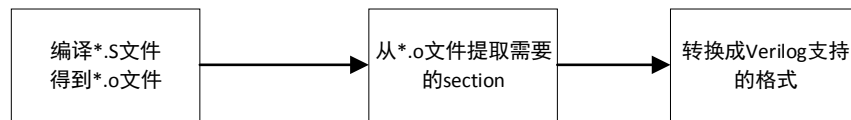
SECTIONS

```
{  
    .text :  
    {  
        *(.text)  
    } > ram  
    .data :  
    {  
        *(.data)  
    } > ram  
    .bss :  
    {  
        *(.bss)  
    } > ram  
}
```

ENTRY (_start)

这个文件的作用是指示链接器按照我们指定的格式生成二进制文件。

首先我们简单介绍一下 make 工具的原理。make 工具被用于自动化的构建过程，它把构建过程分为不同的**目标**，每个目标除了要执行的命令，还会依赖于其他目标。make 工具会分析这些目标的依赖关系，得到正确的编译步骤。对于我们生成固件的过程而言，编译步骤可以简单描述成下面的形式：



分析 Makefile 脚本中各个目标的依赖结构，可以知道它做了下面几件事：

- ① 首先给编译器和链接器加上“mips-linux-gnu-”前缀表示交叉编译；
- ② “all”目标是输入 make 命令后的默认目标，它依赖于 inst_rom.bin，这个文件就是我们最终生成的固件；
- ③ 所有的“.o”目标文件都依赖于“.S”文件，因为我们工程的源代码都是汇编文件。“\$(CC) -mips32 \$< -o \$@"表示使用编译器编译生成目标文件，指令集是 MIPS32；
- ④ inst_rom.om 目标依赖于目标文件和 ram.ld。ram.ld 文件我们已经给出，而目标文件在“.o”目标编译得到；
- ⑤ inst_rom.bin 目标的作用是移除 inst_rom.om 文件中不需要的段(section)并按照 Verilog 格式输出到文件 inst_rom.bin。到此我们就得到了一个可以用于初始化的固件。
- ⑥ clean 目标是清理命令，它会删除所有产出的文件。

下面我们编写一段 MIPS 汇编程序测试一下。在 Makefile 同一个目录下编辑文件 inst_rom.S，输入内容：

```
.org 0x0
.global _start
.set noat
_start:
    lui $1, 0x0101
    ori $1, $1, 0x0101
    ori $2, $1, 0x1100
    or $1, $1, $2
    andi $3, $1, 0x00fe
    and $1, $3, $1
    xori $4, $1, 0xff00
```

```
xor $1, $4, $1
nor $1, $4, $1
```

然后在此目录下执行 make 命令：

```
jasper@DESKTOP-62C17AB:~/rom_src$ make
mips-linux-gnu-as -mips32 inst_rom.S -o inst_rom.o
mips-linux-gnu-ld -T ram.ld inst_rom.o -o inst_rom.om
mips-linux-gnu-objcopy --remove-section .reginfo \
--remove-section .MIPS.abiflags -O verilog inst_rom.om inst_rom.bin
jasper@DESKTOP-62C17AB:~/rom_src$
```

如果 Makefile、rom.ld 或者 inst_rom.S 有任何的语法错误，编译过程就会失败。

打开 inst_rom.bin 文件，它的内容如下：

```
@00000000
3C 01 01 01 34 21 01 01 34 22 11 00 00 22 08 25
30 23 00 FE 00 61 08 24 38 24 FF 00 00 81 08 26
00 81 08 27 00 00 00 00 00 00 00 00 00 00 00 00
```

每 4 个数字是一条指令（32bit），后面的 0 用于对齐到 16 字节。非 0 的部分刚好是汇编程序中的 9 条指令。把这个文件复制到对应位置，然后在 Verilog 代码中输入这个文件的路径。

可以看到 GCC 工具链的 Verilog 格式输出都是以 8bit 为单位。为了能够利用 GCC 工具链自动编译固件，我们就不得不把程序中的 ROM 设计为以 8bit 为最小单位的形式而不是 32 位或者其他。

每当需要新的程序进行测试时，遵循下面的步骤编译生成新的固件：

- ① 编写 inst_rom.S;
- ② 执行 make 命令;
- ③ 如果 make 命令没有出错，那么复制生成的 inst_rom.bin 文件到对应位置；
反之，检查 inst_rom.S 的语法错误

由于汇编程序比 C 语言等更加贴近硬件，导致很多错误不能在编译过程中发现。

例如，写错了指令的参数顺序；计算错了 beq 指令的跳转步长；遗漏了 MIPS 中延迟槽的特性等等。这些情况下编译都不会给出错误或者警告。因而，当你的程序运行的结果和预期不一致时，除了检查 Verilog 部分的代码外，还需要注意是否是汇编程序本身存在错误。

5.1.5 FAQ

问：VMware 下怎么把编译好的文件复制到 Windows 下？

答：在虚拟机中安装 VMware Tools。重启虚拟机后，就可以直接拖拽文件到 Windows 下。

问：WSL 下怎么把编译好的文件复制到 Windows 下？

答：在 WSL 中，C 盘、D 盘等被挂载到 “/mnt” 目录下。这个目录下的 c、d 文件夹就是 Windows 中的磁盘。使用 cp 命令就可以完成文件复制。你还可以把工程目录放在这些文件夹下，这样编译的输出直接可以在 Windows 中访问。

问：执行 make 命令出错了怎么办？

答：仔细检查 Makefile、ram.ld 和汇编源程序。如果自己不能解决的请向助教求助。

问：使用非 Ubuntu 的 Linux 发行版可以吗？

答：理论上可以，但是具体的交叉编译器的命令名字可能有所区别，产出的中间文件可能也不一样（例如多出或缺失某些 section）。你需要修改 Makefile 并检查输出是否和我们需要的固件一致。某些发行版没有预编译的 MIPS 交叉编译环境，你可能需要下载对应的源码进行编译。

问：section 是什么东西？

答：GCC 工具链生成的 *.o 文件的格式是 ELF。ELF 是 Linux 下的可执行文件格式，和 Windows 下的 *.exe 类似 (*.exe 称为 PE 格式)。ELF 格式规定了它的内容由多个段 (section) 构成。我们的汇编程序不需要其他的 section，所以编译

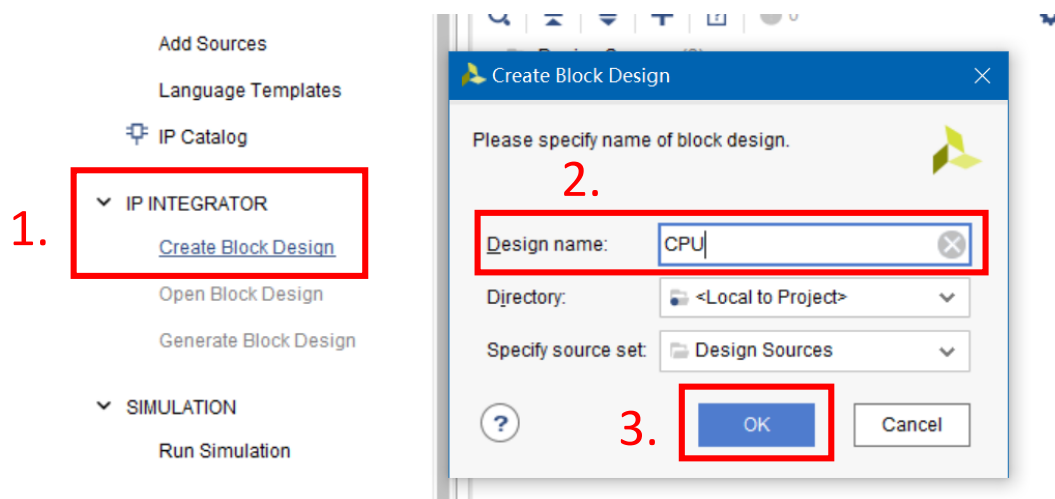
过程中需要去掉多余的 section。

5.2 使用 Vivado 的 Block Design 功能

有时候某个模块的内容只是把其他模块连接起来封装在一起。这种模块可以使用 Vivado 的 Block Design 功能完成。

5.2.1 新建 Block Design

首先点击 Vivado 左侧的 Flow Navigator，选择“IP INTEGRATOR”下的“Create Block Design”，在弹出的窗口里输入模块的名字：



点击“OK”后，就会出现一个新的窗口“Diagram”。在这里可以对模块内容进行编辑。

5.2.2 绘制

首先，右击 Diagram 的空白处，查看这个菜单的功能：

- ① Add IP：添加 IP 核。这里的 IP 核一般是 Vivado 自带的。我们的设计中没有用到。
- ② Add module：添加模块。用 Verilog 编写好其他模块后，就可以用这个功能把它加到我们的 Block Design 中。
- ③ Create Port：创建接口，即 Verilog 中的接口定义部分。使用这个功能时一定要注意接口的 input/output 属性和宽度。

添加模块和创建接口之后，就可以点击拖拽对应的引脚实现连线功能。

注意事项：

- ① Diagram 中用到的模块如果被修改，一定要点击窗口上方出现的“Refresh Changed Modules”。更新之后一般会出现警告，可以不予理会。
- ② 在直接用 Verilog 编写模块时，执行仿真不需要编译。但是使用 Diagram 的情况下，每次仿真前你都需要 Run Synthesis。

5.3 除法器实现

Divider.v

```
1  `timescale 1ns / 1ps
2  `include "global.v"
3
4  `define DivFree      2'b00
5  `define DivByZero    2'b01
6  `define DivOn        2'b10
7  `define DivEnd       2'b11
8
9  module Divider(
10     input clk, input rst,
11     input is_signed,
12     input start, input cancel,
13     input[`DataBus] operand1,
14     input[`DataBus] operand2,
15
16     output reg ready,
17     output reg[`DoubleDataBus] result
```

```
18     );
19
20     reg[5:0] count;
21     reg[64:0] dividend;
22     reg[1:0] state;
23
24     reg[31:0] divisor;
25
26     wire[32:0] div_temp = {1'b0, dividend[63:32]} - {1'b0,
divisor};
27
28     always@(posedge clk) begin
29         if(rst) begin
30             state <= 0;
31             ready <= 0;
32             result <= 0;
33         end
34         else case(state)
35             `DivFree: begin
36                 if(start && !cancel) begin
37                     if(operand2 == 0) begin
38                         state <= `DivByZero;
39                     end
40                     else begin
41                         state <= `DivOn;
42                         count <= 0;
43                         dividend <= 0;
44                         dividend[32:1] <= (is_signed && operand1[31]
```

```

    == 1'b1 ? (~operand1+1): operand1);
45         divisor <= (is_signed && operand2[31] ==
    1'b1 ? (~operand2+1): operand2);
46         end
47     end
48     else begin
49         ready <= 0;
50         result <= 0;
51     end
52 end
53 `DivByZero:begin
54     dividend <= 0;
55     state <= `DivEnd;
56 end
57 `DivOn: begin
58     if(!cancel) begin
59         if(count != 6'b100000) begin
60             if(div_temp[32] == 1'b1) begin
61                 dividend <= {dividend[63:0], 1'b0};
62             end
63             else begin
64                 dividend <= {div_temp[31:0],
dividend[31:0], 1'b1};
65             end
66             count <= count + 1;
67         end
68     else begin // count == 32
69         if(is_signed) begin

```

```
70             if((operand1[31] ^ operand2[31]) ==
1'b1) dividend[31:0] <= (~dividend[31:0] + 1);
71             if((operand1[31] ^ dividend[64]) ==
1'b1) dividend[64:33] <= (~dividend[64:33] + 1);
72             end
73             state <= `DivEnd;
74             count <= 0;
75             end
76         end
77     else begin
78         state <= `DivFree;
79     end
80 end
81 `DivEnd: begin
82     result <= {dividend[64:33], dividend[31:0]};
83     ready <= 1;
84     if(!start) begin
85         state <= `DivFree;
86         ready <= 0;
87         result <= 0;
88     end
89 end
90 endcase
91 end
92 endmodule
```

图例

标题样例

名称	类型	宽度	方向	用途
clk	wire	1	input	时钟信号
rst	wire	1	input	重置信号

标题样例

```
93 `timescale 1ns / 1ps
94 `include "global.v"
95
```